

— EXECUTIVE SECURITY BRIEFING

The Compliance Gate for Code

Entering

Our GitHub Organization

A unified, automated control point that stands between every uploaded project and production — enforcing secrets hygiene, secure-coding standards, supply-chain integrity, and AI-era risk controls before a single line ships.

20+

INTEGRATED ENGINES

6

GATE PHASES

0

SILENT BYPASSES

Every new codebase is a new blast radius — until it passes the gate

Code brought into our organization — by engineers, contractors, or increasingly by AI coding assistants — is the single largest source of unmanaged risk in the software supply chain. Ignite closes that gap with one automated, non-negotiable checkpoint applied to **100% of onboarded projects**.



Breach exposure, cut at the source

Leaked credentials, vulnerable dependencies, and insecure patterns are caught before code ever reaches a repository a threat actor can reach.



Audit-ready by default

Every decision — pass, block, or human-justified override — is attributed, timestamped, and logged. Evidence for SOC 2, ISO 27001, and EU AI Act reviews is a byproduct, not a project.

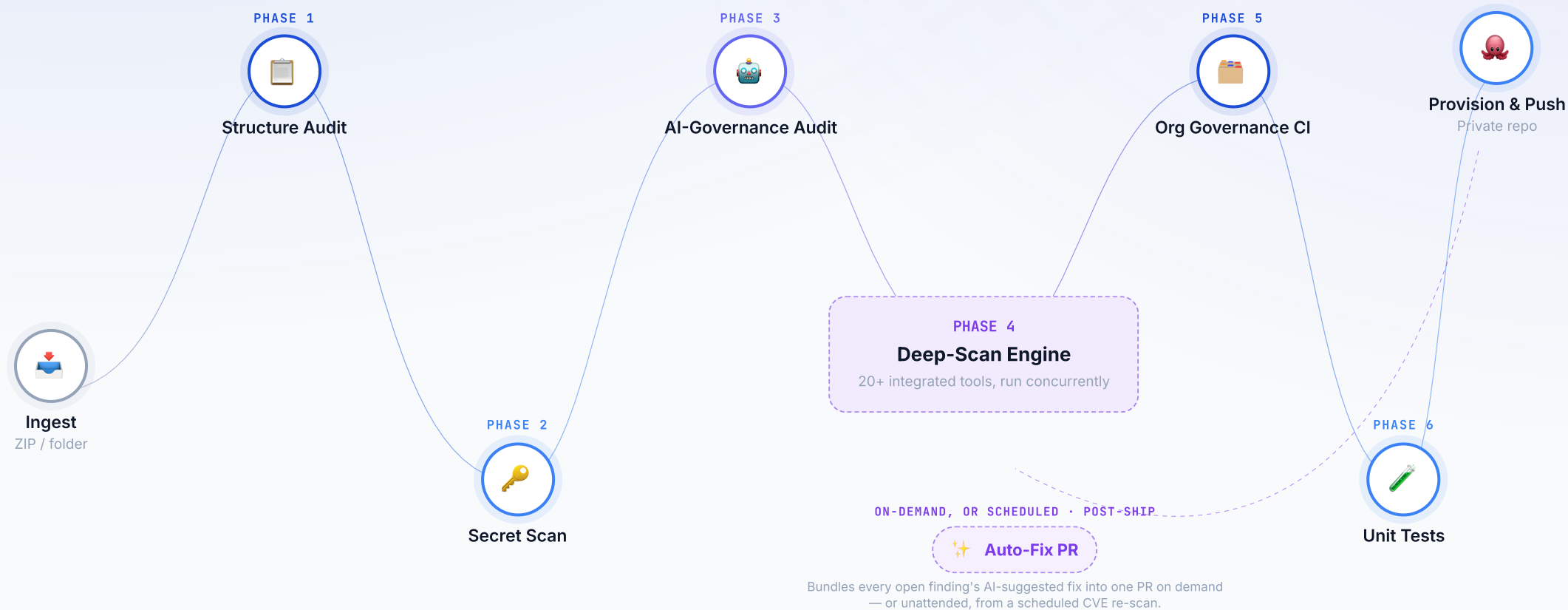


Security that doesn't slow delivery

Fully automated and parallelized — engineering teams get a pass/fail verdict in minutes, with one-click AI-drafted fix PRs instead of a ticket queue.

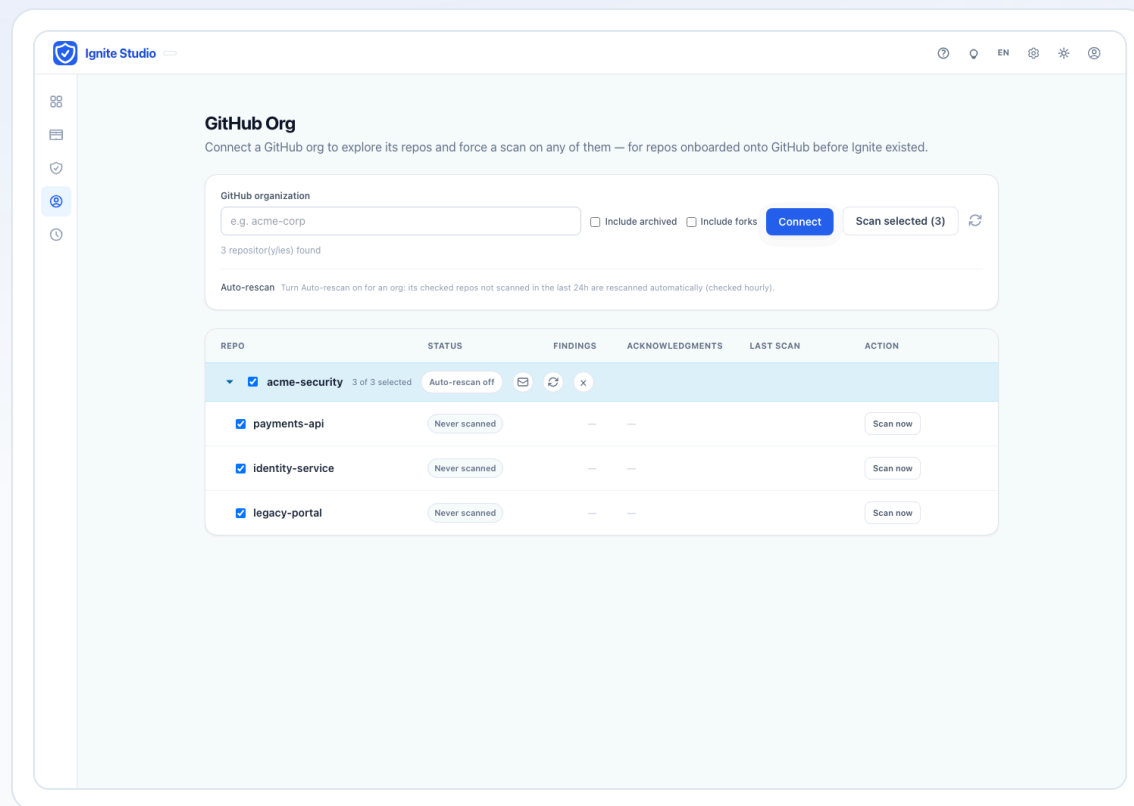
One pipeline, six gates, zero silent exceptions

Every upload flows left to right through independent checkpoints. A single failed gate halts the run — nothing reaches GitHub without every phase clearing, or a justified, attributed override.



The control continues after the first scan

Ignite now gives security teams a working view of the full repository portfolio: backfill the organization, choose continuous coverage, track remediation, and retain evidence for every decision.



Portfolio coverage

Discover existing repositories, select coverage, and scan one or many repositories without a separate onboarding project.

Continuous remediation

Campaigns turn a category of open findings into a measurable, date-bound burn-down goal. Progress reflects the live issue state.

Evidence on demand

Daily reports, compliance audit packs, downloadable findings, and a verified audit chain make review evidence a system output.

Static analysis & secret detection

The first line of code-level defense — every one of these engines is a category leader, not an in-house heuristic.

Semgrep SEMGREP, INC.

OWASP-aligned static-analysis engine covering 30+ languages; the default SAST layer for security-mature engineering orgs and CI/CD pipelines industry-wide.

CodeQL GITHUB / MICROSOFT

Cross-file, cross-function taint-tracking engine powering GitHub Advanced Security — catches injection and IDOR chains single-file scanners miss entirely.

Bearer BEARER.COM

Purpose-built PII / GDPR data-flow analysis — traces sensitive data from source to sink for privacy-risk and regulatory-exposure reporting.

Gitleaks OPEN-SOURCE STANDARD

The most widely deployed open-source secret scanner — entropy + regex detection before a commit ever leaves staging.

Picklescan AI-ERA ADDITION

Opcodes-level scan of ML model artifacts (.pkl/.pt/.bin/.ckpt) for malicious deserialization payloads — a risk vector standard SAST tools were never built to see.

Supply chain, container & infrastructure security

The tooling that closed the SolarWinds- and Log4j-class gap: what's inside the artifact, and can its origin be trusted.

Trivy AQUA SECURITY / CNCF

The de-facto standard for container image, IaC misconfiguration, and dependency vulnerability scanning.

Checkov PALO ALTO NETWORKS

Policy-as-code IaC compliance across Terraform, CloudFormation, and Kubernetes manifests.

Syft & Cosign SIGSTORE / LINUX FOUNDATION

CycloneDX SBOM generation plus Sigstore-backed signature verification for supply-chain integrity.

GuardDog DATADOG SECURITY RESEARCH

Heuristic inspection of npm/PyPI packages for malicious behavior before they're trusted.

hadolint OPEN-SOURCE STANDARD

Dockerfile best-practice linting — unpinned base images, missing non-root USER, caught pre-build.

AI-era risk & compliance governance

Risk vectors that didn't exist five years ago — purpose-built for a world where a meaningful share of committed code is AI-generated or AI-assisted.

Package-Hallucination Guard IGNITE, BUILT-IN

Live registry checks against npm / PyPI / crates.io catch "slopsquatting" — an LLM-invented package name an attacker already registered.

oasdiff OPEN-SOURCE STANDARD

Diffs every API spec against its prior committed revision to catch breaking changes and shadow endpoints.

Spectral STOPLIGHT

OpenAPI / AsyncAPI governance linting against organizational schema standards.

Compliance & Posture Engine IGNITE, ON SEMGREP

Classifies SSO / RBAC / audit-logging / encryption posture, plus EU AI Act Art. 5, 12, 13 & 50 signals.

CodeQL Cross-File Analysis GITHUB / MICROSOFT

The `security-extended` query suite, applied whole-project — closing the gap when a tainted-data chain spans a controller, a service layer, and a template.

zizmor TRAIL OF BITS

Audits every committed GitHub Actions workflow for pwn requests, script injection via untrusted `$` expansions, and over-broad permissions — the CI-pipeline supply-chain risk a source-code scanner never sees.

GHAS is one engine in the gate, not the gate itself

GHAS is genuinely best-in-class for what it does — Ignite runs its CodeQL engine directly. The difference is scope and timing: GHAS secures a repo that already exists; Ignite decides whether the repo gets created at all.

CHECK	 GHAS	 IGNITE
Secret scanning	✓ Push protection + scanning	✓ Gitleaks + custom regex, incl. full git history
Static analysis (SAST)	✓ CodeQL only	✓ CodeQL + Semgrep + Bearer
Dependency vulnerabilities	✓ Dependabot	✓ Trivy + SBOM diffing
IaC & container misconfig.	– Not covered	✓ Trivy, Checkov, hadolint
SBOM & signature verification	 Dependency graph only	✓ Syft SBOM + Cosign signing
PII / GDPR data-flow analysis	– Not covered	✓ Bearer
AI package-hallucination check	– Not covered	✓ Package-Hallucination Guard
Pre-push blocking gate	– PR annotation only	✓ Blocks provisioning until fixed/overridden
Findings in GitHub's native UI	✓ Security & Insights tabs	✓ SARIF + dependency-graph push into the same tabs

Not a replacement for GHAS where it's already licensed — Ignite folds it in as one of twenty-plus engines behind a single go/no-go gate.

From one-time gate to ongoing governance platform

This cycle's additions move Ignite beyond a pass/fail checkpoint at onboarding time, toward continuous, organization-wide visibility — with AI doing more of the verification work automatically.



Security campaigns

Org-wide, named burndown tracking — target a category of risk across every onboarded repo, set a deadline, and watch a live progress bar as it's resolved. Turns "we should fix our X" into a trackable initiative with board-visible progress.



AI-assisted dependency validation

One click in Ignite Studio sends the resolved dependency list to the configured LLM to sanity-check the license/compliance classification — catching a scanner-side misread before it reaches a human reviewer as a false alarm.



Live GitHub webhook sync

Ignite's view of a repo now stays current between scans — a human dismissing an alert or bypassing push-protection directly in GitHub is reflected instantly, and a newly created repo is enrolled and protected automatically, with no manual step.

Compliance audit packs

One export bundles overrides-by-severity, mean-time-to-resolution, and live SLA-breach status for any date range — ready-made evidence for a SOC 2, ISO 27001, or board review, generated on demand rather than assembled by hand.

Closing the remaining GHAS-parity gaps

Three more additions round out the gap analysis from this deck's engine-comparison section — each one a specific capability GitHub Advanced Security customers would otherwise lose.



Active secret verification

A flagged secret is optionally checked read-only against its own provider (GitHub, AWS, GCP, Slack, Stripe, OpenAI, and more) to tell a still-live credential apart from a dead one — off by default, since probing a live token is an operator's explicit call.



Dependency reachability

A known-vulnerable dependency finding is now annotated with whether the flagged package — or, for Go, the exact vulnerable function — is actually used in source, so triage can prioritize real exposure over an unused transitive entry.



PR dependency review comment

Every pull request gets a single, self-updating comment summarizing what dependencies were added, removed, or version/license-changed — the same at-a-glance review GitHub's own dependency-review action provides.

Known false positives, acknowledged by policy — not hidden

Some findings are structurally benign across a whole estate — e.g. SAP integration packages whose `*_Credential_Name=` lines hold alias names, not secrets. Security now declares that once, centrally, instead of asking hundreds of teams to justify the same line.



Scoped org / repo / file rules

A rule targets an organization, a repository (wildcards allowed), file paths, the exact flagged line, and optionally a finding category — narrow enough that a real secret on a different line is still caught.



Pass the gate, keep the evidence

Matched findings are still reported and fully navigable, but arrive pre-acknowledged with the rule's written reason — the repository shows as passing, and an auditor sees exactly what was accepted and why.



Revocable in one edit

Policy acknowledgments are never carried forward on their own: remove or tighten a rule and the next scan re-flags every finding it used to cover. Each one is attributed to the policy, not to a person.

⚙️ Distinct from a human override

Policy acknowledgments carry their own "Org rule" label wherever findings are reviewed — Studio and the organization view — so a board or auditor can always separate centrally-approved exceptions from individual engineers' justifications.

Every finding has an owner — and the owner hears about it

A finding nobody owns doesn't get fixed. Ignite now names who last changed each flagged line and delivers each organization's open findings to the people accountable for it, on a schedule security sets once.



Line-level ownership

Every finding shows who last changed that exact line — not just the last committer to the repository — in review, in reports and in the SIEM feed, so remediation goes straight to the right engineer.



Reports to the org's owners

Each organization's open findings are emailed to its GitHub owners — or to fixed distribution lists, where a business unit prefers a team inbox — with an automatic fallback when no owner can be reached.



Cadence per organization

Weekly for one portfolio, every working day for another: each organization gets its own schedule, defined centrally in configuration and versioned with it — not scattered across personal settings.

Accountability without chasing

Security stops forwarding spreadsheets: owners receive their organization's findings directly, each one traceable to the line and the person who introduced it, while every delivery is recorded for audit.

Deploys as your own product, in your team's language

The onboarding console isn't a third-party vendor tool bolted onto the workflow — every customer-facing surface is themeable and localized out of the box, so it sits inside your organization's tooling as if you built it.



White-label branding

Product name, logo, and accent color are a single configuration file, not code — apply your organization's brand with zero risk of merge conflicts against upstream updates.



Four-language UI

The console ships fully localized in English, French, Portuguese, and German, with a one-click switcher — no redeployment required to change language.



Config, not a fork

Both brand and language are runtime configuration — a customer deployment never diverges from upstream source, so every future Ignite release still applies cleanly on top.

Best-in-class tooling, applied consistently, with a paper trail

No engine in this stack is an unproven internal script — each is either the recognized leader in its category, or a purpose-built Ignite control for a risk class those vendors don't yet cover. The value isn't any single scanner; it's that all of them run **every time, on every project, with no exceptions that aren't logged.**



Consistent enforcement

One gate, zero exceptions for who's uploading — engineer, contractor, or agent.



Regulator-ready evidence

Attributed overrides and full scan history, on demand — no scramble before an audit.



Board-level assurance

A single control the executive team can point to, backed by industry-standard tooling.



Recommendation: keep Ignite as the single mandatory onboarding gate; expand override-review cadence to quarterly board visibility.

END OF BRIEFING